Linguagem de desenho — Fase 1

Especificação: do texto à AST

Integrante	RA
Giulia Nogueira Lopes de Sá	23024383
Ruan Paulo de Sousa Soares	23024986

1. Descrição da linguagem

A linguagem descreve um desenho como uma sequência de comandos. Todo programa começa com a declaração da **tela**, que dá a largura e a altura do desenho. Depois vêm, em qualquer quantidade, **formas** (reta, círculo e retângulo) e **grupos**.

- Os números de cada forma e de cada modificador vão entre parênteses e separados por vírgula, como em uma chamada de função: `circulo(cx, cy, raio)`.
- Os atributos de estilo vêm depois da forma, no formato `nome=valor`: `traco=` (cor do traço), `fundo=` (cor do preenchimento) e `espessura=` (espessura do traço). Todos são opcionais.
- Um grupo é escrito como `grupo`, seguido de zero ou mais modificadores, `girar(ângulo)` e `mover(dx, dy)`, e de um bloco entre chaves `{ }` com as formas e os grupos internos.
- Comentários começam com `%` e vão até o fim da linha.
- Espaços e quebras de linha são livres entre os símbolos; vários comandos podem ficar na mesma linha.

Comando	Argumentos	Significado
`tela(L, A)`	largura, altura	Tamanho do desenho (obrigatório, só no início)
`reta(x1, y1, x2, y2)`	dois pontos	Segmento de (x1, y1) até (x2, y2)
`circulo(cx, cy, r)`	centro e raio	Círculo
`retangulo(x, y, L, A)`	canto superior esquerdo, largura, altura	Retângulo
`grupo ... { ... }`	modificadores e corpo	Reúne formas e outros grupos
`girar(a)`	ângulo	Modificador de grupo: rotação
`mover(dx, dy)`	deslocamento horizontal e vertical	Modificador de grupo: deslocamento

Exemplo 1 — Carro

```
tela(400, 300)
% corpo do carro
retangulo(50, 150, 200, 60) fundo=azul

% roda dianteira, com um raio girado
grupo mover(70, 190) {
    circulo(0, 0, 25) fundo=preto
    grupo girar(45) {
        reta(0, 0, 20, 0) traco=branco espessura=2
    }
}

% roda traseira
grupo mover(200, 190) {
    circulo(0, 0, 25) fundo=preto
}
```

Exemplo 2 — Grupos aninhados e modificadores

```
tela(500, 500)
grupo mover(250, 250) girar(30) {          % deslocamento e depois rotação
    circulo(0, 0, 100) traco=cinza
    grupo girar(90) mover(10, 0) {        % rotação e depois deslocamento
        retangulo(0, 0, 40, 40) fundo=amarelo
        grupo {                            % grupo sem modificadores
            reta(0, 0, 0, 80) traco=roxo espessura=1
        }
    }
}
```

2. Definição léxica

O analisador léxico percorre o texto da esquerda para a direita e, em cada posição, testa as regras **na ordem da tabela**; a primeira que casa forma o token. `ESPACO` e `COMENTARIO` são reconhecidos e **descartados**: não aparecem na lista de tokens.

O sufixo `(?![\wÀ-ÿ])` é um lookahead negativo que significa "aqui termina a palavra": a regra só casa se depois dela não vier letra, dígito ou `_`. Ele evita que uma palavra reservada case com o começo de uma palavra maior (`retas` não vira `RETA` + `s`; vira erro léxico) e faz `05` e `10px` serem erros em vez de vários tokens. `RETANGULO` vem antes de `RETA` na tabela pelo mesmo motivo.

Token	Expressão regular	Descrição
TELA	`tela(?![\wÀ-ÿ])`	Palavra reservada: declaração da tela
RETANGULO	`retangulo(?![\wÀ-ÿ])`	Palavra reservada: forma retângulo
RETA	`reta(?![\wÀ-ÿ])`	Palavra reservada: forma reta
CIRCULO	`circulo(?![\wÀ-ÿ])`	Palavra reservada: forma círculo
GRUPO	`grupo(?![\wÀ-ÿ])`	Palavra reservada: início de grupo
GIRAR	`girar(?![\wÀ-ÿ])`	Modificador de grupo: rotação
MOVER	`mover(?![\wÀ-ÿ])`	Modificador de grupo: deslocamento

Token	Expressão regular	Descrição
TRACO	`traco(?![\wÀ-ÿ])`	Atributo: cor do traço
FUNDO	`fundo(?![\wÀ-ÿ])`	Atributo: cor do preenchimento
ESPESSURA	`espessura(?![\wÀ-ÿ])`	Atributo: espessura do traço
COR_NOME	`(vermelho|verde|azul|amarelo|laranja|roxo|rosa|preto|branco|cinza)(?![\wÀ-ÿ])`	Cor de uma lista fechada de dez nomes
COR_HEX	`#[0-9A-Fa-f]{6}(?![\wÀ-ÿ])`	Cor no formato #RRGGBB
NUM	`(0|[1-9][0-9]*)(?![\wÀ-ÿ])`	Inteiro sem sinal; aceita o zero, sem zeros à esquerda
ABREPAR	`\(`	Abre a lista de argumentos
FECHAPAR	`\)`	Fecha a lista de argumentos
VIRGULA	`,`	Separa argumentos
IGUAL	`=`	Liga o atributo ao valor
ABRECHAVE	`\{`	Abre o corpo do grupo
FECHACHAVE	`\}`	Fecha o corpo do grupo
ESPACO (ignorado)	`\s+`	Espaços, tabulações e quebras de linha
COMENTARIO (ignorado)	`%[^\n]*`	Do % até o fim da linha

Erro léxico: quando nenhuma regra casa, o analisador registra a linha e o trecho inválido, do ponto do erro até o próximo espaço, pula esse trecho e continua. Assim todos os erros léxicos do programa são listados de uma vez. Se houver qualquer erro léxico, a análise sintática não é executada.

3. Definição sintática

Gramática livre de contexto na notação das aulas: palavras em MAIÚSCULAS são tokens, em minúsculas são variáveis, `|` separa alternativas e `ε` é a cadeia vazia. O **símbolo inicial é programa**.

```
programa       → tela elementos
tela           → TELA ABREPAR NUM VIRGULA NUM FECHAPAR
elementos      → elemento elementos | ε
elemento       → forma | grupo
forma          → RETA ABREPAR NUM VIRGULA NUM VIRGULA NUM VIRGULA NUM FECHAPAR estilo
               | CIRCULO ABREPAR NUM VIRGULA NUM VIRGULA NUM FECHAPAR estilo
               | RETANGULO ABREPAR NUM VIRGULA NUM VIRGULA NUM VIRGULA NUM FECHAPAR estilo
estilo         → traco_opc fundo_opc espessura_opc
traco_opc      → TRACO IGUAL cor | ε
fundo_opc      → FUNDO IGUAL cor | ε
espessura_opc  → ESPESSURA IGUAL NUM | ε
cor            → COR_NOME | COR_HEX
grupo          → GRUPO modificadores ABRECHAVE elemento elementos FECHACHAVE
modificadores  → modificador modificadores | ε
modificador    → GIRAR ABREPAR NUM FECHAPAR
               | MOVER ABREPAR NUM VIRGULA NUM FECHAPAR
```

Adequação ao parser descendente recursivo

- **Sem recursão à esquerda:** as repetições (`elementos`, `modificadores`) são recursivas à direita, e no código viram laços `while`.
- **Escolha pelo próximo token:** as alternativas de cada variável começam por tokens diferentes, e cada regra com `ε` tem conjunto FOLLOW disjunto do FIRST da outra alternativa, como mostra a tabela.

Variável	FIRST da alternativa não vazia	FOLLOW (quando escolhe ε)
elementos	RETA, CIRCULO, RETANGULO, GRUPO	FECHACHAVE, fim do programa
traco_opc	TRACO	FUNDO, ESPESSURA, RETA, CIRCULO, RETANGULO, GRUPO, FECHACHAVE, fim
fundo_opc	FUNDO	ESPESSURA, RETA, CIRCULO, RETANGULO, GRUPO, FECHACHAVE, fim
espessura_opc	ESPESSURA	RETA, CIRCULO, RETANGULO, GRUPO, FECHACHAVE, fim
modificadores	GIRAR, MOVER	ABRECHAVE
elemento	forma: RETA, CIRCULO, RETANGULO · grupo: GRUPO	—
modificador	GIRAR · MOVER	—
cor	COR_NOME · COR_HEX	—

Erro sintático: o parser para no primeiro erro e informa a linha, o que era esperado e o que foi encontrado. Quando o programa acaba antes da hora, o encontrado é "o fim do programa" e a linha é a do último token.

```
Linha: 3
Esperado: VIRGULA ","
Encontrado: FECHAPAR ")"
```

4. Árvore sintática abstrata (AST)

A AST só é gerada quando não há erros. Ela guarda os números como números, não guarda pontuação nem delimitadores (parênteses, vírgulas, `=` e chaves somem), reproduz o aninhamento dos grupos e mantém os modificadores na ordem em que foram escritos.

Nó	Campos	O que representam
Programa	tela, elementos	Raiz. `tela` é um nó Tela; `elementos` é a lista de formas e grupos de nível mais alto, na ordem do texto
Tela	largura, altura	Tamanho do desenho
Reta	x1, y1, x2, y2, estilo	Os dois pontos do segmento
Circulo	cx, cy, raio, estilo	Centro e raio
Retangulo	x, y, largura, altura, estilo	Canto superior esquerdo e dimensões
estilo (objeto da forma)	traco, preenchimento, espessura	Cores como texto (`azul`, `#3A7D44`) e espessura como número; `null` quando o atributo não foi escrito

Nó	Campos	O que representam
Grupo	modificadores, elementos	Lista ordenada de modificadores e lista de formas e grupos internos
Rotacao	angulo	Modificador `girar`
Deslocamento	dx, dy	Modificador `mover`

AST do Exemplo 1 (Carro)

Visão em árvore, como a página mostra:

```
Programa
├─ Tela 400 x 300
├─ Retangulo (50, 150) 200 x 60 [preenchimento azul]
├─ Grupo
│  ├─ Deslocamento (70, 190)
│  ├─ Circulo centro (0, 0) raio 25 [preenchimento preto]
│  └─ Grupo
│     ├─ Rotacao 45
│     └─ Reta (0, 0) até (20, 0) [traço branco, espessura 2]
└─ Grupo
   ├─ Deslocamento (200, 190)
   └─ Circulo centro (0, 0) raio 25 [preenchimento preto]
```

A mesma AST em JSON:

```
{
  "tipo": "Programa",
  "tela": {
    "tipo": "Tela",
    "largura": 400,
    "altura": 300
  },
  "elementos": [
    {
      "tipo": "Retangulo",
      "x": 50,
      "y": 150,
      "largura": 200,
      "altura": 60,
      "estilo": {
        "traco": null,
        "preenchimento": "azul",
        "espessura": null
      }
    },
    {
      "tipo": "Grupo",
      "modificadores": [
        {
          "tipo": "Deslocamento",
          "dx": 70,
          "dy": 190
        }
      ],
      "elementos": [
        {
          "tipo": "Circulo",
          "cx": 0,
```

```
          "cy": 0,
          "raio": 25,
          "estilo": {
            "traco": null,
            "preenchimento": "preto",
            "espessura": null
          }
        },
        {
          "tipo": "Grupo",
          "modificadores": [
            {
              "tipo": "Rotacao",
              "angulo": 45
            }
          ],
          "elementos": [
            {
              "tipo": "Reta",
              "x1": 0,
              "y1": 0,
              "x2": 20,
              "y2": 0,
              "estilo": {
                "traco": "branco",
                "preenchimento": null,
                "espessura": 2
              }
            }
          ]
        }
      ]
    },
    {
      "tipo": "Grupo",
      "modificadores": [
        {
          "tipo": "Deslocamento",
          "dx": 200,
          "dy": 190
        }
      ],
      "elementos": [
        {
          "tipo": "Circulo",
          "cx": 0,
          "cy": 0,
          "raio": 25,
          "estilo": {
            "traco": null,
            "preenchimento": "preto",
            "espessura": null
          }
        }
      ]
    }
  ]
}
```

5. Decisões de projeto

Questão	Decisão	Exemplo e resultado
Marcador de comentário	`%` até o fim da linha. Não usamos `#` nem `//`: o `#` é o início das cores hexadecimais, e um marcador diferente elimina a ambiguidade.	`traco=#FFFFFF % borda` → cor + comentário
Zero e zeros à esquerda	`0` é aceito. Um número com zero à esquerda é **erro léxico** (a ER do NUM não o aceita e o lookahead impede quebrar em `0` e `5`).	`espessura=05` → erro léxico "05"
Palavra que começa com palavra reservada	Erro léxico: as palavras reservadas só casam como palavra inteira.	`retas(0,0,1,1)` → erro léxico
Números	Inteiros sem sinal. Sinal, ponto decimal e unidade são erros léxicos.	`-1`, `2.5`, `10px` → erro léxico
Tela	Obrigatória, exatamente uma, sempre como primeiro comando.	Programa sem `tela` ou com duas → erro sintático
Programa sem formas	Permitido: o resultado é uma tela em branco.	`tela(200, 100)` → válido
Grupo vazio	**Não** é permitido: a gramática exige pelo menos um elemento dentro das chaves.	`grupo { }` → erro sintático
Atributos repetidos	**Não** são permitidos. Cada atributo aparece no máximo uma vez, na ordem fixa `traco`, `fundo`, `espessura`. A gramática garante isso com as variáveis opcionais em sequência.	`traco=azul traco=verde` → erro sintático
Ordem dos atributos	Fixa (traço, preenchimento, espessura), pelo mesmo motivo.	`fundo=azul traco=verde` → erro sintático
Modificadores	Zero ou mais, só em grupos, entre `grupo` e `{`. Podem repetir (`mover` duas vezes) e a ordem escrita é mantida na AST, porque na Fase 2 girar e depois mover é diferente de mover e depois girar.	`grupo mover(5,5) girar(30) { ... }`
Modificador em forma	Não permitido.	`reta(0,0,1,1) girar(9)` → erro sintático
Cores	Dez nomes fixos em minúsculas e #RRGGBB com dígitos hexadecimais maiúsculos ou minúsculos. Na AST a cor fica como foi escrita.	`#12ab` → erro léxico
Maiúsculas	As palavras reservadas e as cores nomeadas são em minúsculas.	`Tela(1,1)` → erro léxico
Valores fora de faixa	Não são verificados nesta fase (tela 0 x 0, raio 0, ângulo 720). Isso é semântica, da Fase 2.	`circulo(0,0,0)` → válido
Atributos ausentes	Ficam `null` no objeto `estilo` da AST, para a Fase 2 aplicar o padrão.	—

6. Implementação

JavaScript puro, com módulos, sem bibliotecas de análise, a partir do tutorial do Robô.

Arquivo	Conteúdo
`tokens.js`	Lista de regras léxicas (tipo, expressão regular, se é ignorada)
`lexer.js`	`analisarLexico(texto)`: devolve os tokens (tipo, lexema, linha) e os erros léxicos
`parser.js`	`analisarSintatico(tokens)`: parser descendente recursivo, uma função por variável da gramática; devolve a AST ou lança `ErroSintatico` com linha, esperado e encontrado
`main.js`	Liga a página às duas fases e traz os casos de teste no seletor
`index.html`	Página de teste: caixa de texto, botão, tabela de tokens, erros ou AST (árvore e JSON)
`casos-de-teste.md`	5 programas válidos e 3 inválidos com o resultado esperado de cada um